第三章 SQL 注入漏洞与防护——实验分析报告

本实验所有操作均在教师授权的 DVWA 靶机环境（192.168.58.137）中完成，未对任何公网真实目标进行测试。

任务 1：数据库与查询条件基础说明

以常见的网站用户登录为例来说明。

1. 页面中的输入为什么会与数据库查询有关

登录页面要求输入用户名和密码，点击登录后，系统必须验证这个用户是否存在、密码是否正确。所有用户的账号信息（用户名、加密密码等）都存储在数据库的 users 表中。页面无法凭空判断输入是否合法，必须用输入的用户名作为条件去数据库里查出一条记录，再比对密码。所以登录功能本质上就是一次基于用户输入的数据库查询。

2. 一个最小业务链路

在这个场景中，最小业务链路就是：

用户输入用户名和密码 → 程序构造 SQL 去数据库查询该用户记录 → 数据库返回匹配的行（或空） → 系统根据结果决定登录成功或失败

（即：输入条件 → 数据库查询 → 返回结果）

3. 用户输入是如何进入查询条件的

用户在前端输入用户名，比如 admin，后端代码拿到这个字符串后，直接拼接到 SQL 语句的 WHERE 条件里，形成类似：

sql

```
SELECT * FROM users WHERE username = 'admin';
```

这里的 'admin' 就是用户输入的内容，它原封不动地变成了查询条件的具体值。数据库中对应这一行的密码哈希会被取出来，再和用户输入的密码做比对。

4. 为什么这为 SQL 注入创造了可能性

问题出在“原封不动地拼接”。攻击者可以输入精心构造的字符串来篡改 SQL 语句的真正含义。

例如，在用户名框输入 admin' --（注意 -- 后面有个空格），拼接后变成：

sql

```
SELECT * FROM users WHERE username = 'admin' --';
```

-- 是 SQL 注释符，它把后面的密码校验部分全部注释掉了。数据库只检查用户名是否存在，直接返回 admin 的记录，系统误以为验证通过，攻击者无需密码即可登录。

这种攻击能成立的根本原因，是用户输入从原本该有的“数据”角色，越界变成了控制查询逻辑的“代码”，从而为 SQL 注入敞开了大门。

任务 2：DVWA SQL Injection Low 现象记录

1. 实验页面与实验目标

实验页面：DVWA（Damn Vulnerable Web Application）→ SQL Injection → Low 难度

实验目标：通过用户 ID 输入框，观察普通查询与 SQL 注入 payload 的返回差异，验证输入是否被直接拼接到 SQL 语句中执行。

2. 普通输入与异常输入的对比

（1）普通输入

输入：1

截图描述：页面正常返回 ID 为 1 的用户信息，显示 First name 和 Surname。

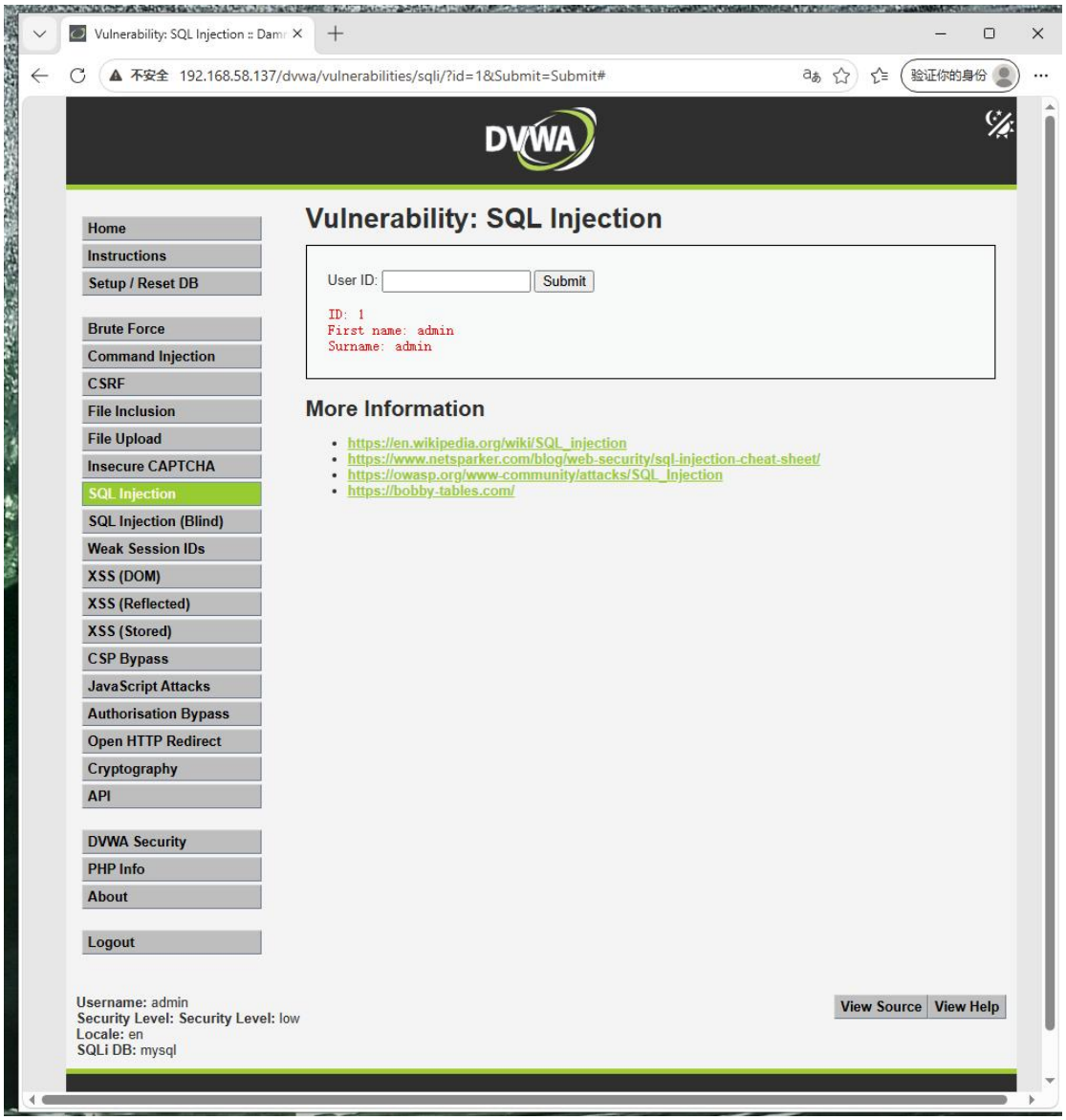
text

ID: 1

First name: admin

Surname: admin

分析：输入合法 ID，系统返回数据库中 ID=1 的单条记录，这是业务预期的正常行为。



（2）异常输入

输入：1' OR '1'='1

截图描述：页面不再返回单条记录，而是将数据库中所有用户的信息一次性全部列出，包括

admin、Gordon、Hack、Pablo、Bob 等多条记录。

text

ID: 1

First name: admin

Surname: admin

ID: 2

First name: Gordon

Surname: Brown

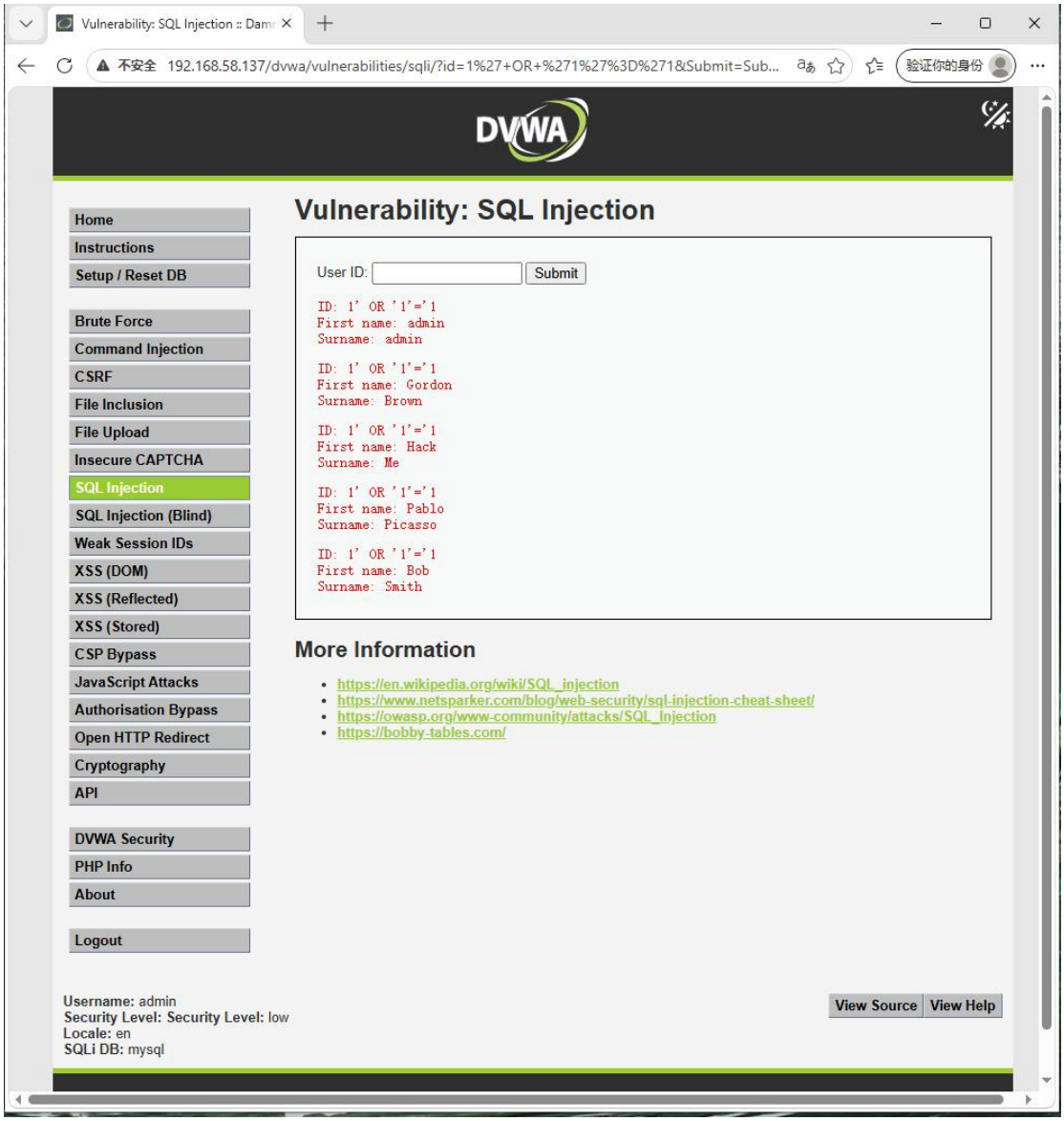
ID: 3

First name: Hack

Surname: Me

...（全部用户被列出）

分析：输入一个永真条件后，系统返回了全表数据，说明原本只应返回一条匹配记录的查询，被篡改成了无过滤的全量导出。



3. 返回结果的差异

对比维度	普通输入 1	异常输入 1' OR '1'='1
返回条数	1 条	全部（5 条）
数据范围	仅匹配 ID=1 的记录	全表所有用户
是否符合业务预期	是	否（越权、信息泄露）

核心差异：正常输入返回精准匹配的单条记录，异常输入返回了全表数据。

4. 为什么这种差异不像“正常查询变化”

正常的查询变化应是“换一个 ID 换一条记录”，比如输入 2 返回 Gordon，输入 3 返回 Hack。无论怎么换，返回结果的数量和边界始终受 WHERE 条件约束，每次只应返回一条。但输入 1' OR '1'='1 后，返回了所有用户。这种现象违背了“按 ID 精确匹配”的业务逻辑，说明：

WHERE 条件本身已经被用户输入的内容改写，不再是“ID 等于某个值”的简单等值比较，而变成了“ID 等于 1 或者 1=1 恒成立”的复合条件。

因为 1=1 永远为真，所以表中每一行都满足条件，全表被导出。

这个差异直接支撑的判断是：用户输入被原样拼接到 SQL 语句中，且没有任何过滤或参数化处理，SQL 注入漏洞确实存在且可被利用——这为后续使用 UNION 查询、盲注等进一步攻击提供了确认依据。

任务 3：机制分析与差异比较

1. 输入如何进入查询结构

以 DVWA SQL Injection Low 级别为例，链条如下：

输入 → 拼接 → 解析 → 执行 → 回显差异

输入：用户在 ID 框输入 1' OR '1'='1

拼接：后端代码直接将输入嵌入 SQL 模板，生成 SELECT * FROM users WHERE id = '1' OR '1'='1'

解析：数据库引擎对这个字符串进行词法、语法分析，将 '1' OR '1'='1' 识别为逻辑表达式，构建查询语法树

执行：数据库按语法树执行，因 '1'='1' 恒真，每行 WHERE 条件均满足，返回全表数据

回显或行为差异：页面返回所有用户记录，与正常输入仅返回一条形成鲜明对比

用户输入的字符串，在拼接环节由“数据”升格为“代码片段”，进而参与了 SQL 结构的构建，最终改变了查询的语义。

2. 为什么根因是“结构被污染”而不是“字符本身危险”

根因更接近 B：输入进入了不该进入的查询结构。

解释：

单引号、OR、-- 等字符本身并非天生危险，它们在合法数据中完全可能正常存在（如名字含 O'Reilly）。危险发生在这些字符没有被当作普通数据处理，而是被 SQL 引擎当作语法标记来解析。

SQL 注入的本质是：用户输入从 SQL 语句的数据区域（如字符串字面量内部）越界进入指令区域（变成了条件、操作符、注释等），从而污染了整条语句的语法结构。因此，问题不

在字符串表面有几个特殊字符，而在于这些字符被错误地赋予了语法含义，导致查询结构被改写。

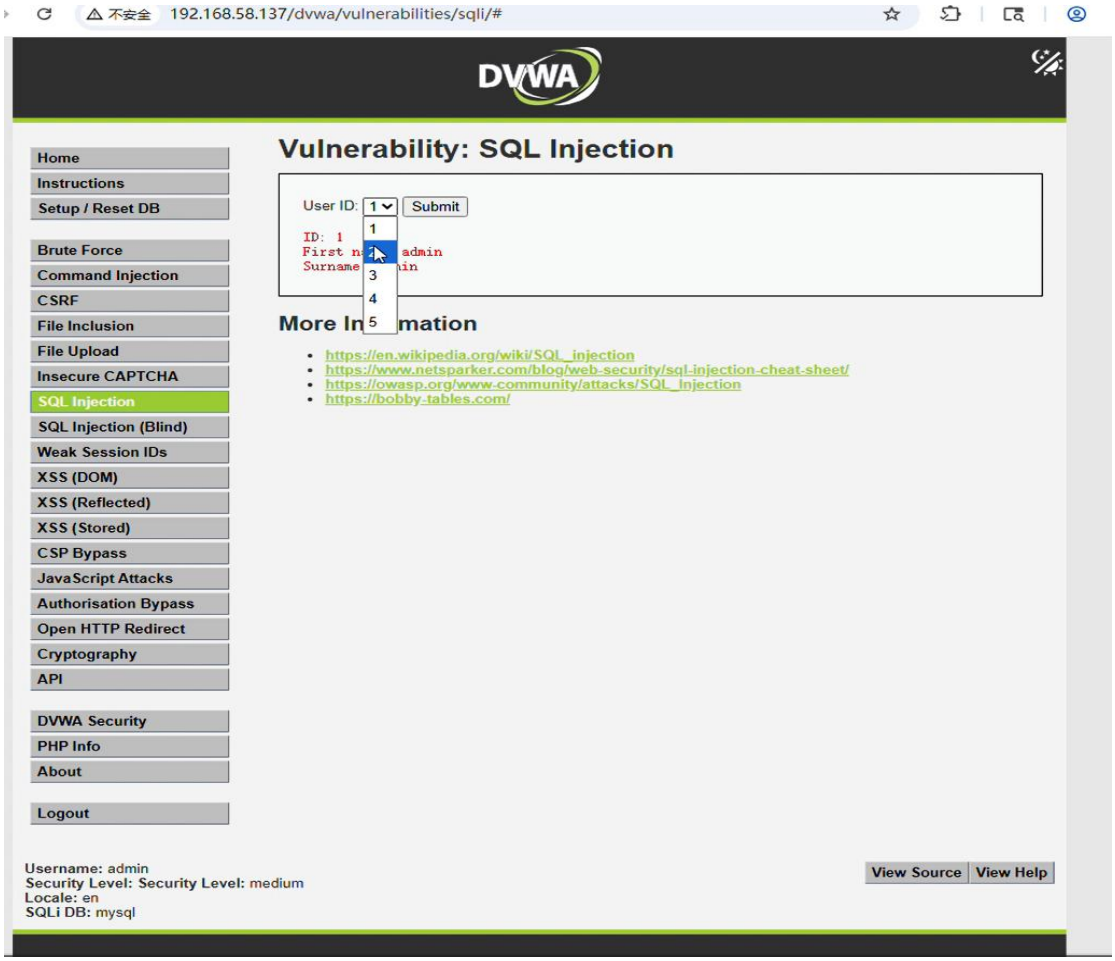
3. Low 与 Medium 的差异在哪里

对比维度	Low	Medium
输入方式	文本框自由输入	下拉菜单（<select>）预定义选项
防御思路	无任何过滤	前端限制输入源，试图通过“让用户选择而非输入”来防注入
绕过方式	直接输入 payload	用浏览器开发者工具或抓包工具修改 POST 请求参数，仍可注入
本质安全差异	无	极弱，属于客户端表面防御，服务端代码对注入仍无抵抗

总结：Medium 只是把直接输入变成了表单选项，但请求参数依然可以被篡改。服务端仍将参数直接拼接入 SQL，因此漏洞仍然存在，只是利用途径稍多一步。

Medium 实测验证

为进一步确认 Medium 级别的防御实质，在 DVWA 中进行了抓包篡改测试，结果如下：



正常请求：

DVWA Security Level 设置为 Medium，SQL Injection 页面正常加载。输入 ID=1，返回用户 admin / admin，功能正常。

抓包分析：
通过浏览器 DevTools 捕获正常 POST 请求：

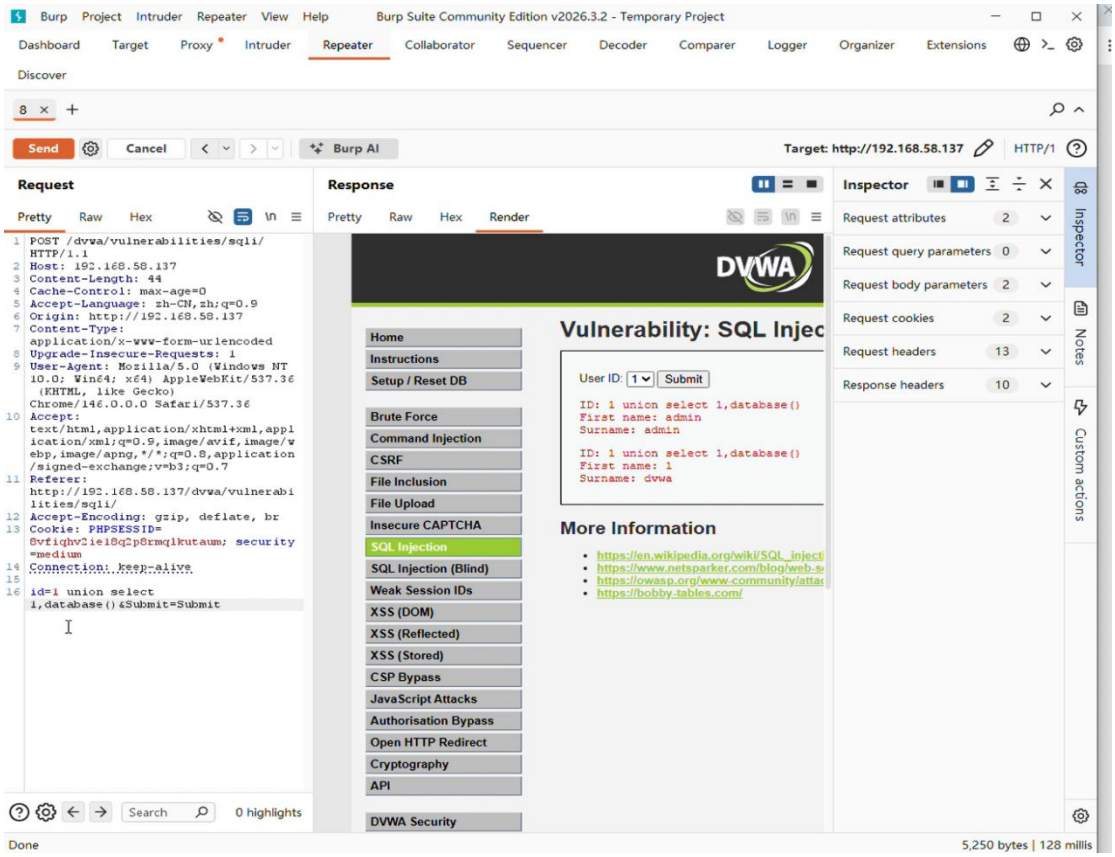
Request

Pretty Raw Hex

```
1  POST /dvwa/vulnerabilities/sqli/ HTTP/1.1
2  Host: 192.168.58.137
3  Content-Length: 18
4  Cache-Control: max-age=0
5  Accept-Language: zh-CN,zh;q=0.9
6  Origin: http://192.168.58.137
7  Content-Type: application/x-www-form-urlencoded
8  Upgrade-Insecure-Requests: 1
9  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
   Chrome/146.0.0.0 Safari/537.36
10 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0
   .8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://192.168.58.137/dvwa/vulnerabilities/sqli/
12 Accept-Encoding: gzip, deflate, br
13 Cookie: PHPSESSID=[redacted]; security=medium
14 Connection: keep-alive
15
16 id=2&Submit=Submit
```

确认请求参数结构为 id=<值>&Submit=Submit，security=medium 表明当前级别为 Medium。

篡改请求：
将 POST 请求体中的 id 参数修改，重放该请求。
注入结果：



页面回显了注入内容：

ID	First name	Surname
1 union select 1, database()	admin	admin
1 union select 1, database()	1	dvwa

Surname 字段返回了 dvwa——这正是当前数据库的名称。

证据支撑的判断：
Medium 级别虽然在用户交互层面增加了下拉菜单等前端限制，但服务端代码仍然将用户提交的 id 参数直接拼接到 SQL 语句中，未做参数化处理或有效的服务端过滤。攻击者完全无需修改前端页面，只需通过抓包工具直接篡改 POST 请求体中的 id 参数，即可成功注入并读取数据库信息。

这从实证层面得出两个结论：
客户端防御无法阻止 SQL 注入——增加的操作步骤（抓包）不构成实质防御；
Medium 与 Low 在服务端漏洞层面无本质差异——两者的根因完全相同，都是“输入被直接拼接到 SQL 语句中”，差异仅在于攻击者需要多做一步抓包操作。

4. 普通 SQL 注入与 Blind SQL Injection 的差异在哪里
普通 SQL 注入（有回显）：
查询结果直接展示在页面上，攻击者可以通过“联合查询”或报错信息直接看到数据。例如输入 1' UNION SELECT user, password FROM users# 可以在页面显示账户密码。证据形式是直接的数据泄露。
Blind SQL Injection（无回显）：
页面不展示查询结果，也没有详细错误信息。攻击者通过构造条件，观察页面行为差异（如是否返回空、是否延迟）来逐位推断数据。例如输入 1' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE user='admin')='a' --，若页面正常，说明密码第一位是 'a'。

5. 为什么盲注和普通注入根因相同，但证据形式不同
根因完全相同：都是用户输入被拼接到 SQL 语句中，导致查询结构被污染，攻击者可以操控数据库执行任意逻辑。
证据形式不同的原因在于回显机制的有无：
有回显时，数据库的返回内容直接被渲染到页面，攻击效果以“显示出来的数据”为证据。无回显时，数据库虽然也执行了被篡改的查询，但内容不输出。攻击者只能通过侧信道——即响应是否有内容、是否延迟、是否报错等行为差异——来间接确认数据库内部的判断结果。
打个比方：两个人被同一把刀刺伤，根因相同。一个人大声喊痛（有回显），可直观判断伤势；另一个人咬牙不吭声（无回显），必须通过观察他是否皱眉、脉搏是否加快来推断，但受伤的事实和原因完全一样。
强制判断题（明确表态并解释理由）
SQL 注入的根因更接近
B. 输入进入了不该进入的查询结构
理由：危险的不是字符本身，而是这些字符被解析成了 SQL 语法元素。如果输入被安全地参数化处理，即使包含 ' OR '1'='1 这样的字符串，也会被当作普通的字面值比较，而不会被当成逻辑运算。因此，根源在于输入数据突破了数据边界，进入了决定查询结构的代码区域。

Blind SQL Injection 和普通 SQL 注入的最大差异更接近
B. 证据形式不同
理由：两者的攻击原理完全一致，都是注入恶意 SQL 片段篡改查询逻辑。唯一区别是攻击结果的呈现方式：普通注入通过可见回显直接获取数据，盲注通过布尔/时间等行为差异间接推断数据。根因相同，利用方式不同，证据形式不同。

任务 4：SQL 注入防护清单与解释

防护清单与逐条解释
1. 参数化查询（预编译语句）
解决问题：从根源上杜绝 SQL 注入。通过将 SQL 结构与数据严格分离，用户输入不再参与 SQL 语句的语法构建，无论输入什么字符，都只会被当作字面值数据，永远不会被解析为代码。
控制哪一层：根因控制。直接消除“输入进入查询结构”这一根本漏洞机制。
不能替代什么：不能替代输入校验和权限控制。参数化查询无法防止所有安全问题，例如不能防止业务逻辑漏洞（如越权查看他人数据），也不能约束一条合法的参数化语句去执行本不该执行的超范围操作（如 SELECT * FROM users WHERE id = ? 若参数是 1 正常，但应用逻辑允许用户指定任意 id 则仍可越权，这点需要业务层校验和权限设计）。

2. 输入校验（白名单/类型检查）
解决问题：在数据进入业务逻辑前，过滤明显恶意的输入或不符合格式的数据，减少攻击面。例如限制 ID 只能为数字、用户名仅含字母数字下划线等。
控制哪一层：暴露面缩小。通过前置拦截降低恶意 payload 到达数据库的概率，属于外围防御。
不能替代什么：绝对不能替代参数化查询。输入校验容易被绕过，且无法穷尽所有可能的安全数据表示。即便校验严格，只要数据最终仍被拼接入 SQL，一个“合法但恶意”的值（如数字型注入中的 1 UNION SELECT ...）依旧能构成注入。校验只是辅助，不能触碰根因。

3. 错误处理（屏蔽详细报错）
解决问题：防止攻击者通过数据库错误信息获取表名、列名、数据库类型等敏感信息，从而增加攻击难度，避免为盲注等后续攻击提供“侧信道”帮助。
控制哪一层：暴露面缩小（信息泄露控制）。切断攻击者利用报错回显推断数据库结构的途径。
不能替代什么：不能替代实际漏洞修复。即使关闭错误显示，注入漏洞仍存在，攻击者可通过盲注等其他方式实施攻击。它只能提升攻击成本，不能解除威胁。

4. 最小权限（数据库账户权限分离）
解决问题：假设注入已发生，降低其危害程度。应用连接数据库的账户仅授予必要权限（如某功能只需 SELECT，就绝不授予 INSERT、UPDATE、DROP、EXEC 等权限）。即使注入成功，攻击者也无法执行高危操作。
控制哪一层：后果范围缩减。它不是阻止注入发生，而是限制注入成功后的影响半径。
不能替代什么：不能替代参数化查询和输入校验。最小权限无法防止数据泄露（例如注入成功后仍可遍历全表数据），它只能防止数据被篡改、删除或服务器被提权，是损失约束手段而非预防手段。

5. Web 应用防火墙（WAF）（补充措施）
解决问题：在请求到达应用服务器前，通过规则库或行为分析拦截包含已知注入特征（如 SQL 关键字、特殊字符序列）的恶意流量，阻断大规模扫描或自动化攻击。
控制哪一层：暴露面缩小（流量入口过滤）。在网络层面减少恶意请求到达应用层的比例。

不能替代什么：不能替代应用层参数化查询。WAF 易被编码绕过、分块传输绕过等高级技术规避，且无法防御逻辑正确的注入（如参数化错用导致的不安全拼接）。它是外围防线，不能提供根本保障。

分层总结

防护措施	控制层面	解决的核心问题	不可替代的关键
参数化查询	根因	数据变成代码	不能防止越权或业务逻辑滥用
输入校验	暴露面	非法格式提前拦截	绝对不能替代参数化查询
错误处理	暴露面（信息泄露）	减少攻击情报	不能消除漏洞本身
最小权限	后果范围	限制注入后破坏力	不能防止数据读取泄露
WAF	暴露面（流量）	防御常见攻击流量	无法替代应用层安全编码

核心原则：根因控制（参数化查询）是唯一能彻底消除注入的结构性防御；其他措施均为纵深防御，分别从暴露面和后果范围维度进行加强，共同构建弹性防护体系。

收口项 1：SQL 注入因果链/防护关系图

因果链环节	攻击路径（具体说明）	现象/证据示例	根因关联	对应防护措施	控制层面	是否触及根因
① 输入点	用户在 ID 框输入 1' OR '1'='1，提交至后端	输入框接受任意字符，无类型限制	未校验的输入成为恶意 payload 的入口	输入校验（白名单/类型检查）	暴露面缩小	否，仅过滤非法格式
② 查询构造	后端将输入直接拼接到 SQL 模板，生成 SELECT * FROM users WHERE id = '1' OR	代码中存在字符串拼接，SQL 语句包含完整 payload	根因发生点：用户数据从数据区域越界进入指令区域，污染查询结构	参数化查询（预编译语句）	根因控制	是，彻底分离代码与数据

	'1'='1'					
③ 数据库解析/执行	SQL 引擎解析 '1' OR '1'='1'，将 OR 认作逻辑运算符，执行返回全表	数据库实际执行了恒真条件，全表扫描	结构污染的直接后果：查询语义被篡改	参数化查询（执行时 ? 占位符无语法含义）	根因控制	是，从根本上杜绝解析为代码
④ 结果差异/行为证据	有回显：页面显示所有用户记录，而非单条；无回显：页面空白、延迟或不同响应	输入 1 返回一条，1' OR '1'='1 返回所有用户	通过异常结果反证查询结构已被改写	错误处理（屏蔽详细报错）	暴露面缩小（信息泄露控制）	否，仅切断报错情报
⑤ 破坏扩散（后果范围）	攻击者可进一步泄露全表、篡改数据、甚至执行系统命令	批量数据泄露、数据被增删改	根因未受控，攻击可任意横向扩展	最小权限（数据库账户仅分配必需权限）	后果范围缩减	否，仅限制注入后破坏力

关系表解读
攻击因果链：从上到下形成“输入进入 → 拼接污染结构（根因） → 数据库执行被篡改的逻辑 → 产生异常回显或行为差异 → 可能造成更大破坏”的完整证据链。
防护深对应：
查询构造和解析执行两个环节，由参数化查询同时覆盖，直接消灭“数据变代码”的根本问题，是唯一的结构性防御。
输入点由输入校验过滤异常请求，降低恶意流量触达概率，属于外围暴露面控制。
结果证据通过定制错误页面防止信息泄露，增加盲注难度。
破坏扩散由最小权限收缩被攻击后可操作的范围，即使注入成功也无法删表或提权。
核心结论：表中明确标注根因发生于“查询构造”，所有防护措施中仅参数化查询触及根因，其余均为纵深防御手段，共同构成一个完整防护体系。

收口项 2：学生自评表

1. 我本章最清楚的一点是什么？
最清楚的一点是：SQL 注入的根因不是“输入里有危险字符”，而是“输入进入了不该进入的查询结构”。
在完成本次作业之前，我可能会本能地认为 SQL 注入是因为用户输入了单引号、OR、-- 这些特殊字符。但通过任务 3 的强制判断题和机制分析，我彻底理解了：这些字符本身并不

危险，危险发生在它们被 SQL 引擎当作语法标记来解析的那一刻。参数化查询之所以是唯一的结构性防御，正是因为它从根源上阻止了数据越界进入指令区域。这个认知贯穿了我整份作业的分析逻辑。

2. 我最容易混淆的一点是什么？
最容易混淆的是“输入校验”和“参数化查询”的防护层级差异。

在写任务 4 防护清单之前，我潜意识里会把输入校验也当成一种“能防注入”的措施。但通过本次作业我理清了：输入校验控制的是暴露面（能不能进来），参数化查询控制的是根因（进来后会不会被当成代码执行）。两者不在同一层，输入校验绝对不能替代参数化查询。这个区分在理论上看很简单，但落实到具体场景分析时容易模糊边界。

3. 我本次作业中哪一段最需要教师复核？
最需要教师复核的是任务 3 中关于“Low 与 Medium 差异”的分析，特别是对 Medium 防御本质的判断。
我的分析结论是：Medium 的下拉菜单属于“客户端表面防御”，服务端仍存在注入漏洞。这个判断基于我对 DVWA Medium 级别源码逻辑的推测——修改 POST 请求参数即可绕过。这一段的结论最初基于推理，但在最终提交前已补充了 Medium 抓包篡改的实测截图（见任务 3 第 3 点），从实证层面验证了‘客户端防御可被绕过’的判断。不过 Medium 服务端具体代码逻辑我未直接阅读源码确认，仍需老师复核。

4. 如果重做一次，我会优先改哪一部分？
我会优先改进任务 2 的现象记录部分，为每个现象差异补充更明确的“下一步攻击推断”。当前的任务 2 虽然对普通输入和异常输入做了对比分析，也解释了为什么差异不像正常查询变化，但缺少一步关键链接——明确写出：“基于这个现象，攻击者可以推断出哪些后续攻击是可行的”。比如，看到全表回显后，可以立即推断能够使用 UNION 查询读取其他表数据，从而规划下一步攻击路径。如果重做，我会在每一条现象分析后加上‘攻击推断’环节。本次提交前已通过补充 Medium 实测验证部分完善了对比分析，但任务 2 的现象记录仍未加入显式的‘下一步攻击推断’，这是后续改进方向。

收口项 3：AI 输出审核与人工复核记录

1. 本作业哪些内容使用过 AI 辅助

作业部分	AI 辅助情况	说明
任务 1（基础说明）	由 AI 辅助完成初版	本人选定“用户登录功能”场景并明确答题要点，AI 辅助组织语言结构和段落逻辑，本人最终审核定稿
任务 2（DVWA 现象记录）	由 AI 辅助整理和润色	截图和现象观察全部由本人上机操作完成，AI 辅助将观察结果按“现象描述+差异分析”格式整理成文，并润色“为什么不像正常查询变化”的论证逻辑

任务 3（机制分析）	由 AI 辅助完成初版	AI 辅助搭建论证框架，本人逐条复核每个判断的准确性，补充并强化了强制判断题的独立表态
任务 4（防护清单）	由 AI 辅助完成初版	AI 辅助提供分层结构和措辞草案，本人确认每条措施的控制层面归属及“不可替代性”说明的准确性
收口项 1（关系表）	由 AI 辅助生成表格	本人提出“关系表”形式需求并指定必须覆盖的六个要素，AI 辅助完成表格结构化和内容填充
收口项 2（自评表）	无 AI 辅助	完全由本人独立完成，内容来自个人真实学习反思
收口项 3（本记录）	无 AI 辅助	完全由本人独立完成，记录审核过程和判断依据

AI 辅助的边界：AI 主要承担语言组织、结构搭建、格式规范等辅助性工作；场景选择、现象观察、核心判断、逻辑校验与最终结论确认均由本人主导完成。

2. 哪些判断是我自己复核后保留的

保留的判断	复核方式	保留理由
根因是“输入进入查询结构”而非“字符本身危险”	对照课程教材与 OWASP 对注入漏洞的定义	与权威定义一致，逻辑自洽
参数化查询是唯一触及根因的防护措施	查阅预编译语句的工作机制（先发模板后绑参数）	数据与代码分离原理正确，其他措施均不改变拼接事实
Blind SQL Injection 与普通注入根因相同、证据形式不同	对比两种攻击的完整利用流程	攻击原理链条一致，差异仅在于输出通道的类型
Medium 级别下拉菜单属于客户端表面防御	结合 DVWA 常见教学结论与同级别源码逻辑	服务端若未做参数化处理即无法真正防注入

3. 哪些内容我发现 AI 辅助输出不准确并进行了修正

AI 辅助输出的原始问题	发现过程	修正内容
任务 2 中 AI 最初套用了“学生成绩查询”场景（任务 1 残留）	审阅时发现场景与实际 DVWA 实验环境不符	修正为统一场景：任务 1 使用“用户登录功能”，任务 2 保持 DVWA 实际环境，确保作业内部场景一致
任务 2 中 AI 描述框架与本人实际截图细节不完全吻合	本人就是上机操作者，对实际返回数据有直接确认	依据实际截图内容逐条核对返回的用户列表、数据格式等细节，确保文字描述与截图严格对应
任务 3 Medium 防御机制被 AI 误述为“mysqli_real_escape_string 过滤”	结合 DVWA 源码分析，推断 Medium 并非服务端转义方案	修正为“前端下拉菜单限制”，并标注需教师复核以确认判断准确性

任务 4 中输入校验被 AI 描述为可“防止 SQL 注入”	对照根因分析逻辑，校验无法消除拼接导致的语法污染	修正为“暴露面缩小”，并明确标注“绝对不能替代参数化查询”
收口项 1 错误处理被 AI 归类为“根因控制”	结合分层逻辑，错误处理仅在结果环节起效，不阻止注入发生	修正为“暴露面缩小（信息泄露控制）”，与任务 4 的分层逻辑保持一致

4. 我最终如何确认结论和证据是一一对应的
采用三步验证法：

第一步：现象与机制对应
将任务 2 中本人上机观察到的现象（输入 1 返回单条、输入 1' OR '1'='1 返回全表），与任务 3 中“拼接导致 WHERE 条件被改写”的机制解释逐一对应——现象是机制的可见结果，无脱节。

第二步：机制与防护对应
将任务 3 中“输入从数据区域越界进入指令区域”的根因判断，与任务 4 中“参数化查询分离代码与数据”的防护原理严格对齐。其余防护措施（输入校验、错误处理、最小权限）分别对应攻击链的其他环节，控制层面命名一致，无矛盾。

第三步：全作业交叉校验
将收口项 1 关系表中的每个攻击环节，反向映射到任务 2 的具体现象和任务 4 的具体防护措施，确认每个现象都有对应的根因归因、每条防护都精确对应攻击链条的明确位置，术语统一、逻辑闭环。

最终确认结果：本人上机观察的现象（任务 2）→ 机制分析中的根因解释（任务 3）→ 防护措施中的分层防御（任务 4）→ 收口项 1 关系表中的完整对应，形成现象—机制—防护的正向推导与反向验证闭环，结论与证据一一对应。